ARGO CD GITOPS › MY LEARNING NOTES › SESSION 03

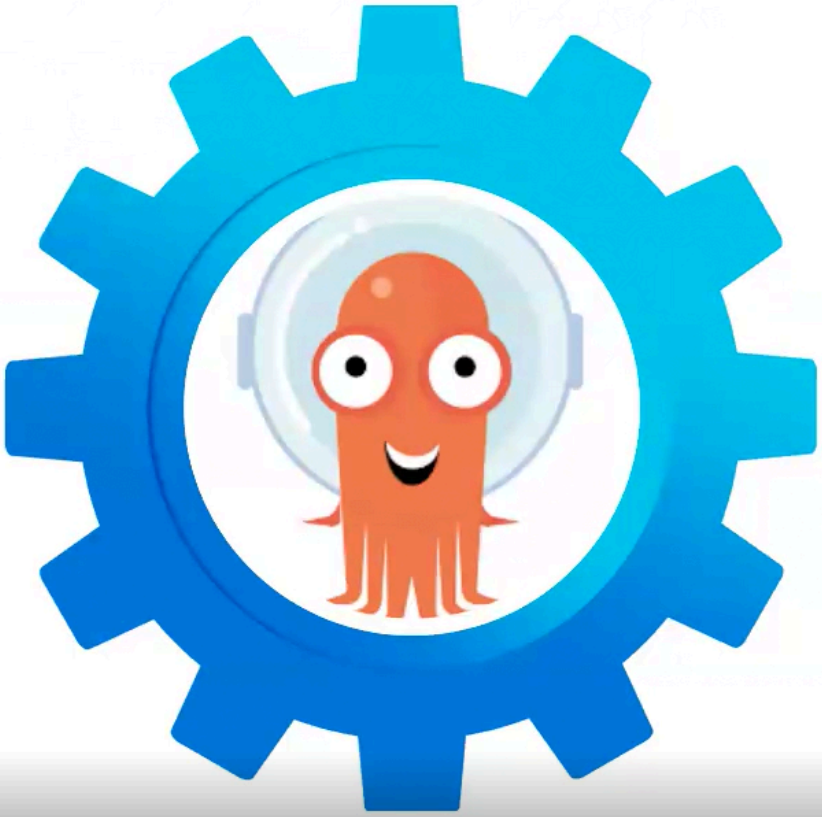
Screenshots: Sync Policies

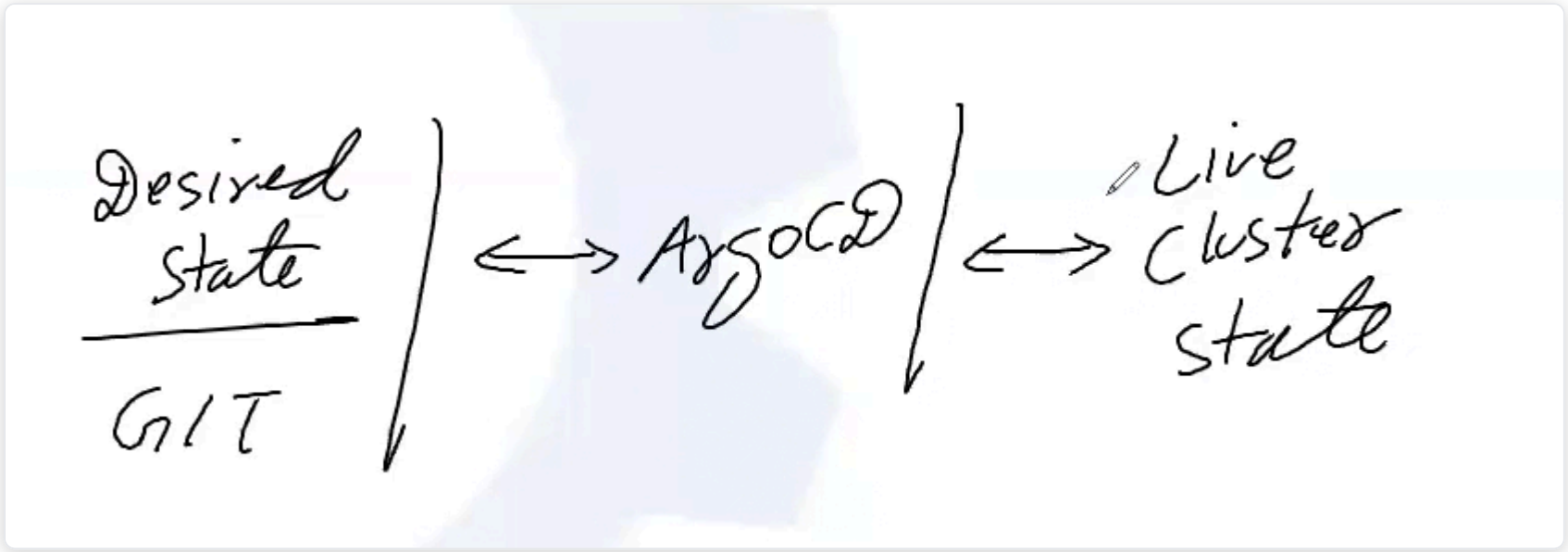
Raw screenshots captured while watching this session (manual vs automatic sync, health status, pruning, self-healing, retry), kept alongside the written notes (03-sync-policies.md) for visual reference.

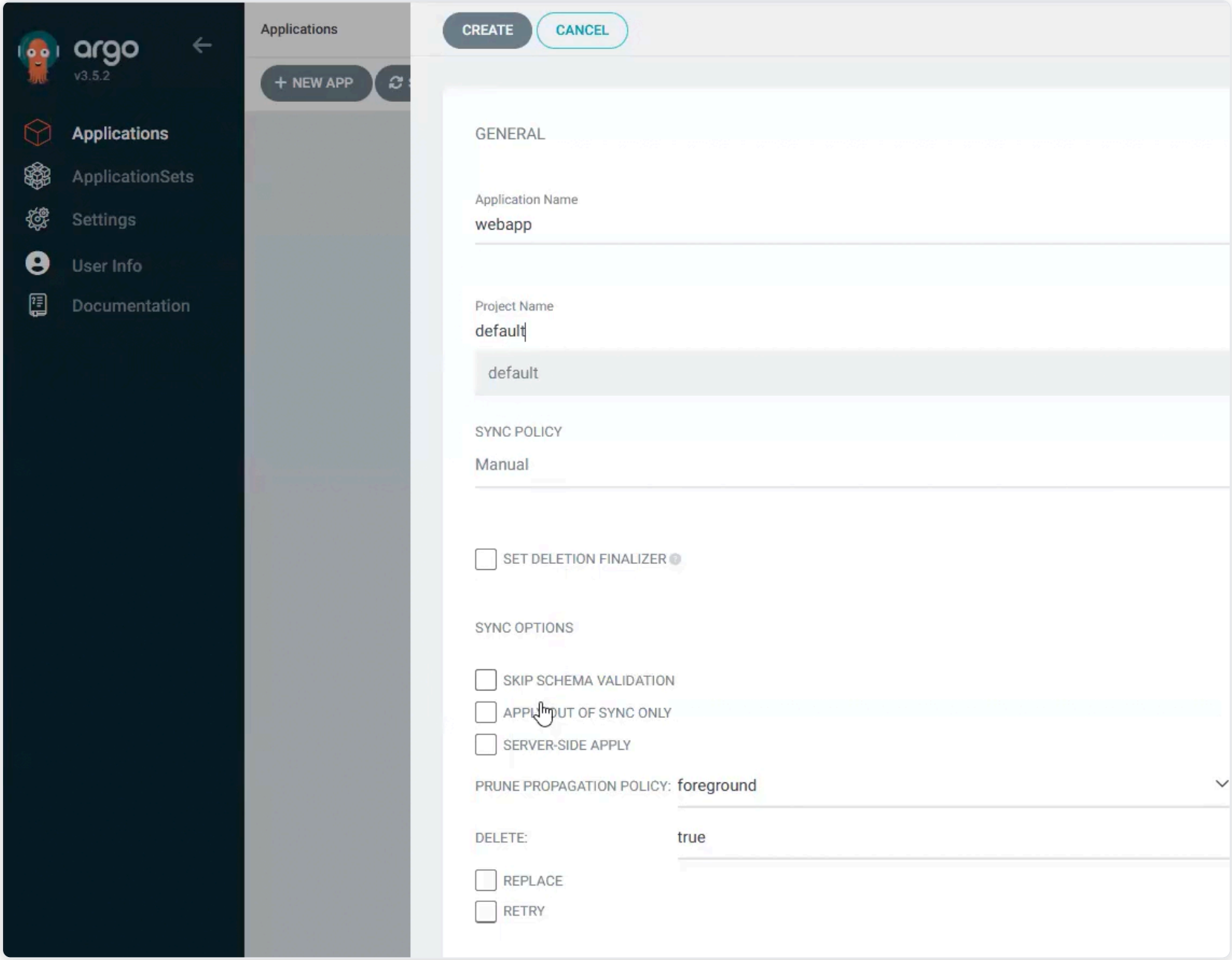
32 screenshots · captured 2026-09-12, 1:37 PM - 3:23 PM

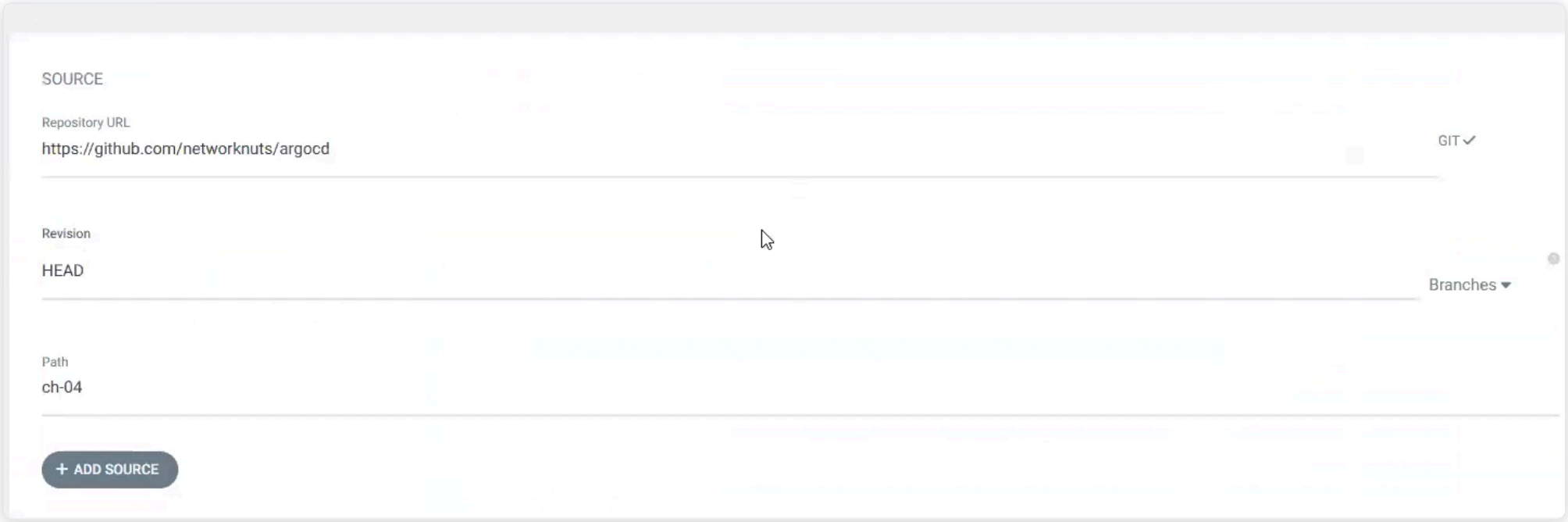
SYNC POLICIES

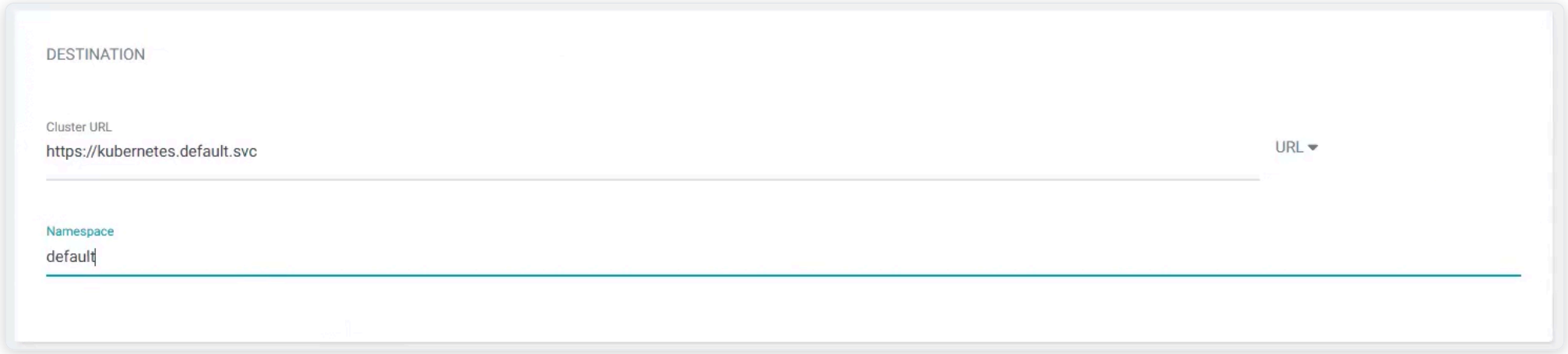
ARGO CD + KUBERNETES

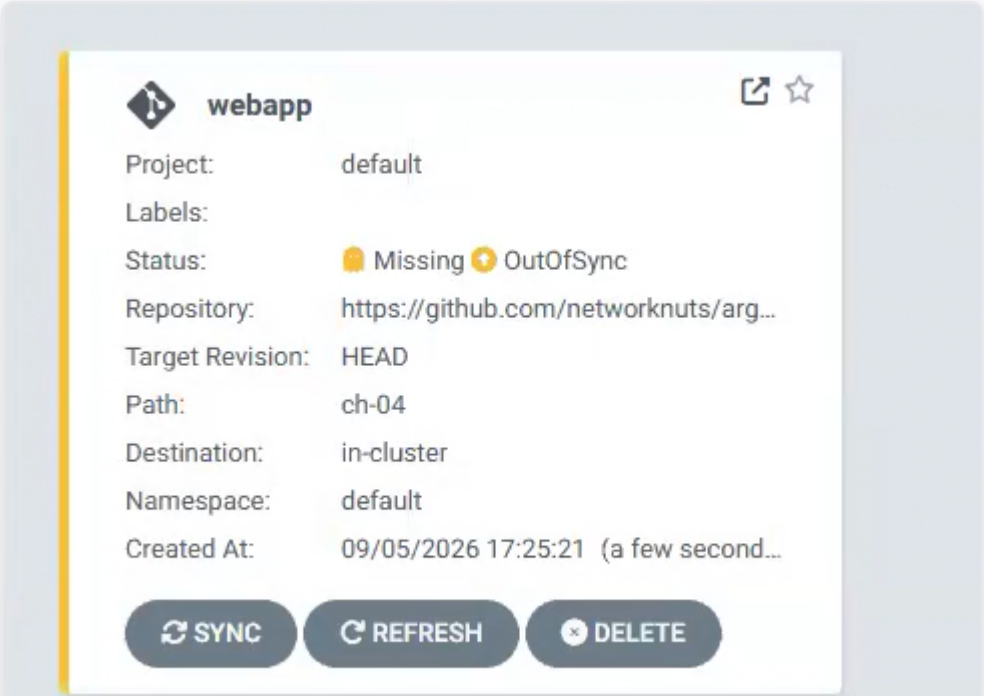


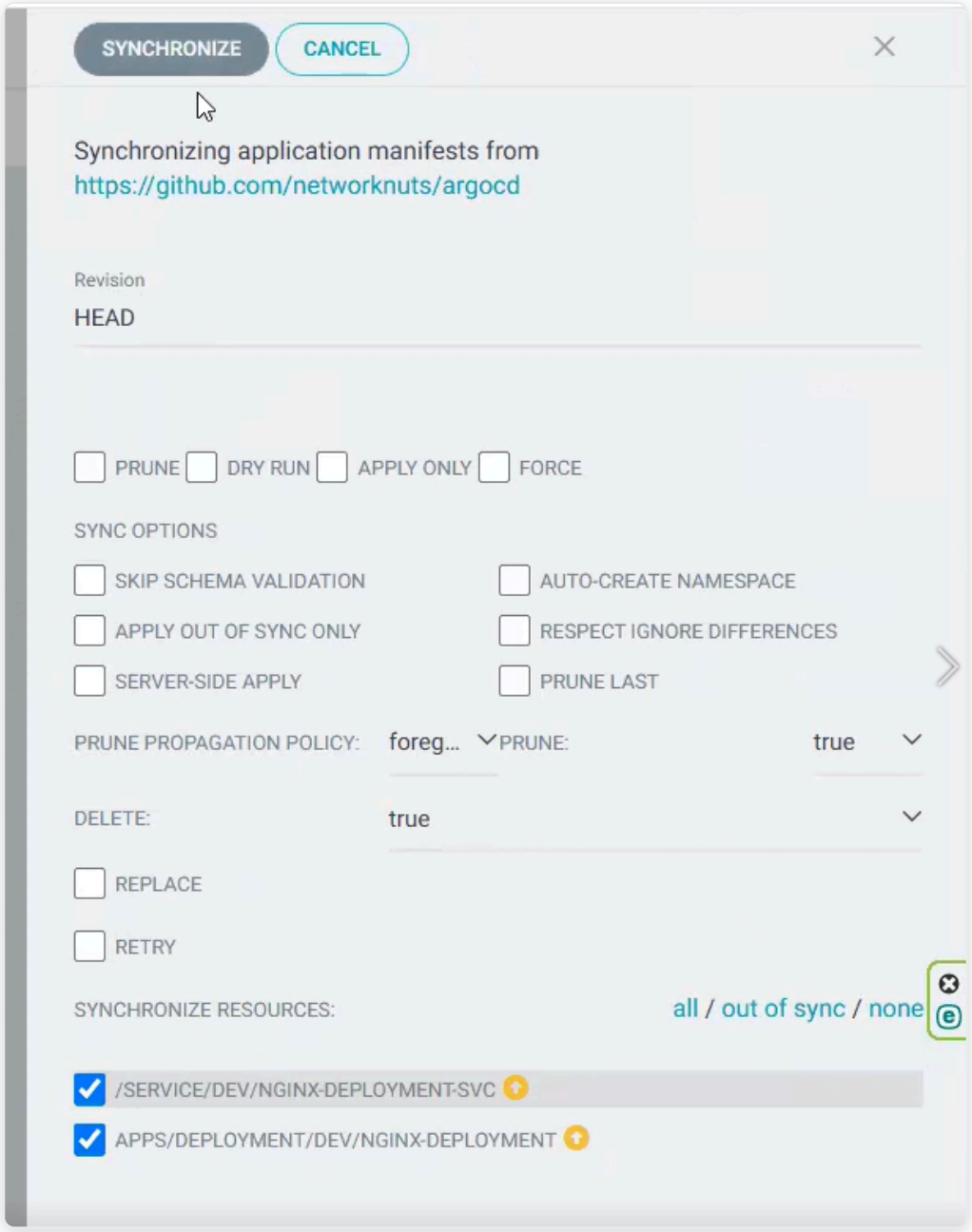

Desired state
GIT
ArgoCD
Live cluster state

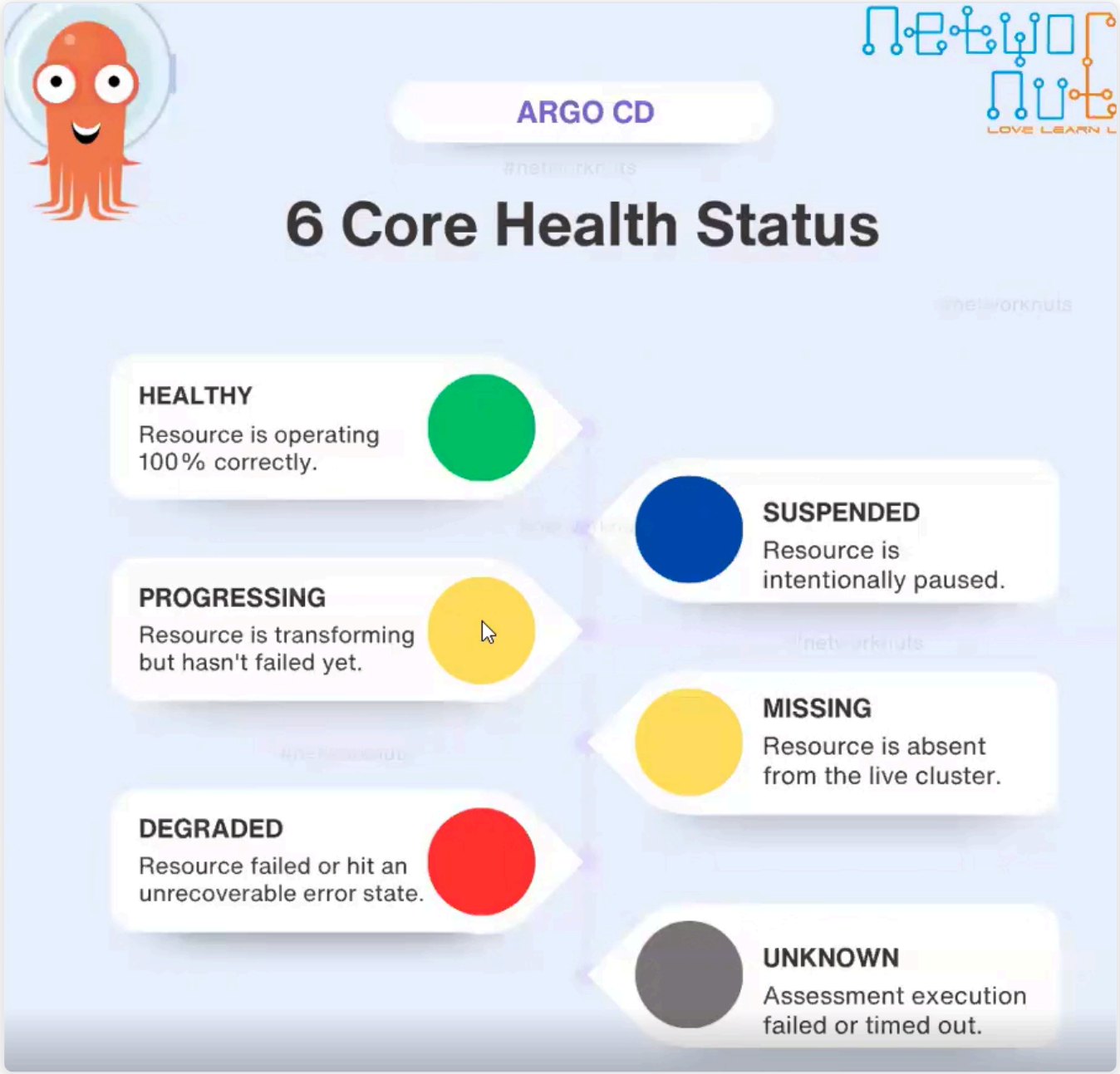

argo
v3.5.2
Applications
ApplicationSets
Settings
User Info
Documentation
Applications
+ NEW APP
CREATE
CANCEL
GENERAL
Application Name
webapp
Project Name
default
default
SYNC POLICY
Manual
SET DELETION FINALIZER
SYNC OPTIONS
SKIP SCHEMA VALIDATION
APPLY OUT OF SYNC ONLY
SERVER-SIDE APPLY
PRUNE PROPAGATION POLICY: foreground
DELETE: true
REPLACE
RETRY


SOURCE
Repository URL
https://github.com/networknuts/argocd
GIT
Revision
HEAD
Branches
Path
ch-04
+ ADD SOURCE


DESTINATION
Cluster URL
https://kubernetes.default.svc
URL
Namespace
default


webapp
Project: default
Labels:
Status: Missing OutOfSync
Repository: https://github.com/networknuts/arg...
Target Revision: HEAD
Path: ch-04
Destination: in-cluster
Namespace: default
Created At: 09/05/2026 17:25:21 (a few second...
SYNC
REFRESH
DELETE


SYNCHRONIZE
CANCEL
Synchronizing application manifests from
https://github.com/networknuts/argocd
Revision
HEAD
PRUNE
DRY RUN
APPLY ONLY
FORCE
SYNC OPTIONS
SKIP SCHEMA VALIDATION
AUTO-CREATE NAMESPACE
APPLY OUT OF SYNC ONLY
RESPECT IGNORE DIFFERENCES
SERVER-SIDE APPLY
PRUNE LAST
PRUNE PROPAGATION POLICY:
foreg...
PRUNE:
true
DELETE:
true
REPLACE
RETRY
SYNCHRONIZE RESOURCES:
all / out of sync / none
/SERVICE/DEV/NGINX-DEPLOYMENT-SVC
APPS/DEPLOYMENT/DEV/NGINX-DEPLOYMENT


ARGO CD
6 Core Health Status
HEALTHY
Resource is operating 100% correctly.
PROGRESSING
Resource is transforming but hasn't failed yet.
DEGRADED
Resource failed or hit an unrecoverable error state.
SUSPENDED
Resource is intentionally paused.
MISSING
Resource is absent from the live cluster.
UNKNOWN
Assessment execution failed or timed out.

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: newapp
  namespace: argocd # The namespace where Argo CD is installed
spec:
  project: default # Default project provided by Argo CD
  source:
    repoURL: 'https://github.com/networknuts/argocd.git' # Your Git repo URL
    targetRevision: HEAD # Git branch, tag, or commit hash
    path: ch-04 # Path within the repository containing the YAML files
  destination:
    server: 'https://kubernetes.default.svc' # Target cluster URL (this means the local cluster)
    namespace: red# Target namespace where resources will be deployed
  syncPolicy:
    automated: # Enables automated sync and self-healing
      prune: true # Deletes resources from K8s if removed from Git
      selfHeal: true # Overwrites manual changes made directly in the cluster
~
~
~
~
~
```

Argo CD Sync Policies - Role

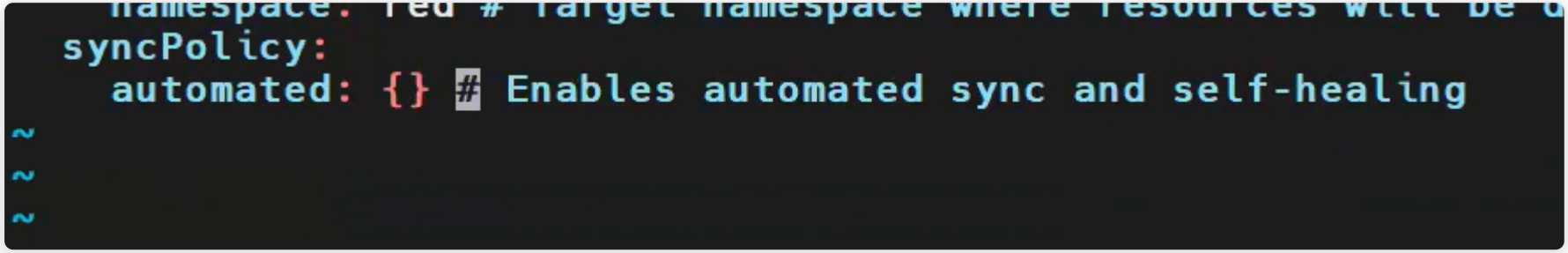
If you are creating the application from an application manifest, you can specify a sync policy in the spec:syncPolicy attribute:

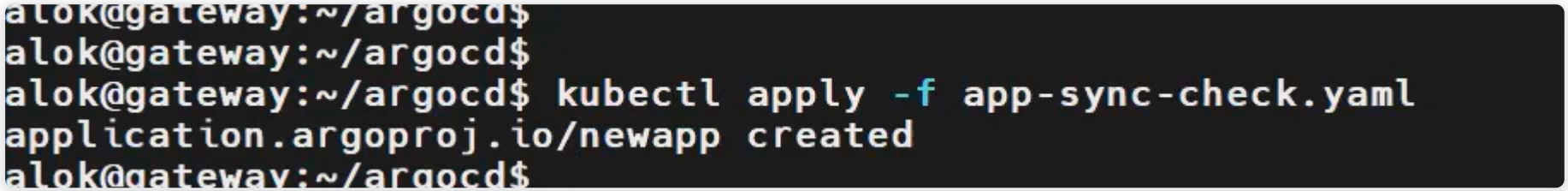
```
spec:
  syncPolicy:
    automated: {}
```

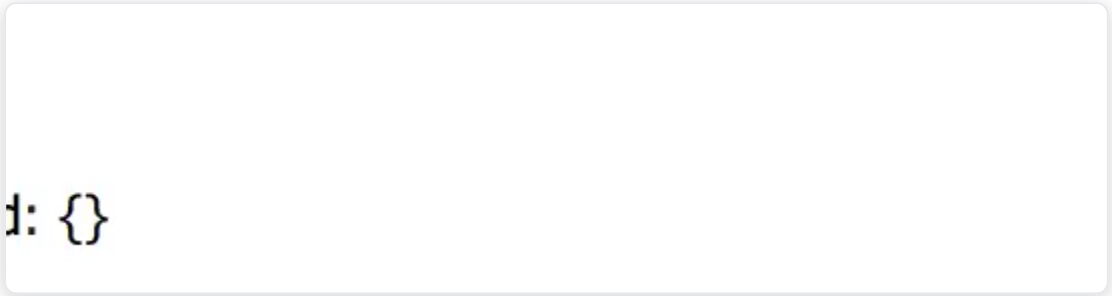
You can also use Argo CD CLI:
argocd app set APP-NAME --sync-policy automated

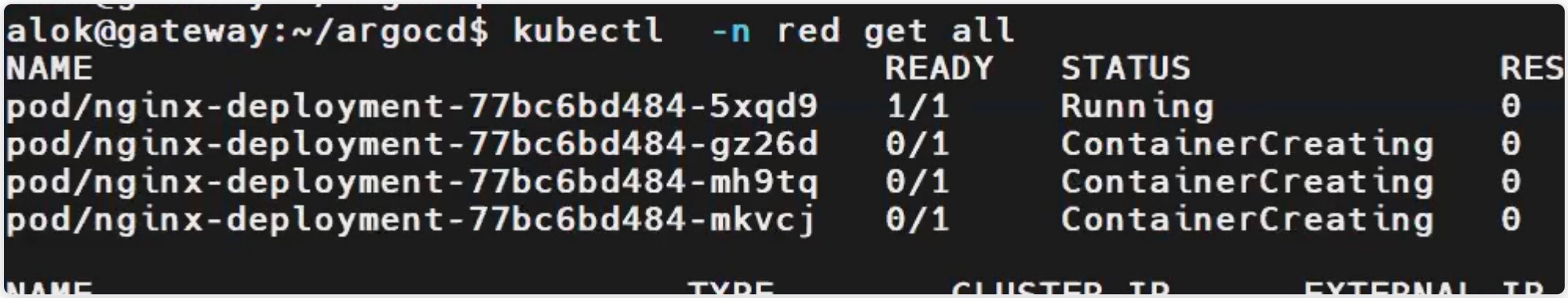
Basic automation triggers sync when:

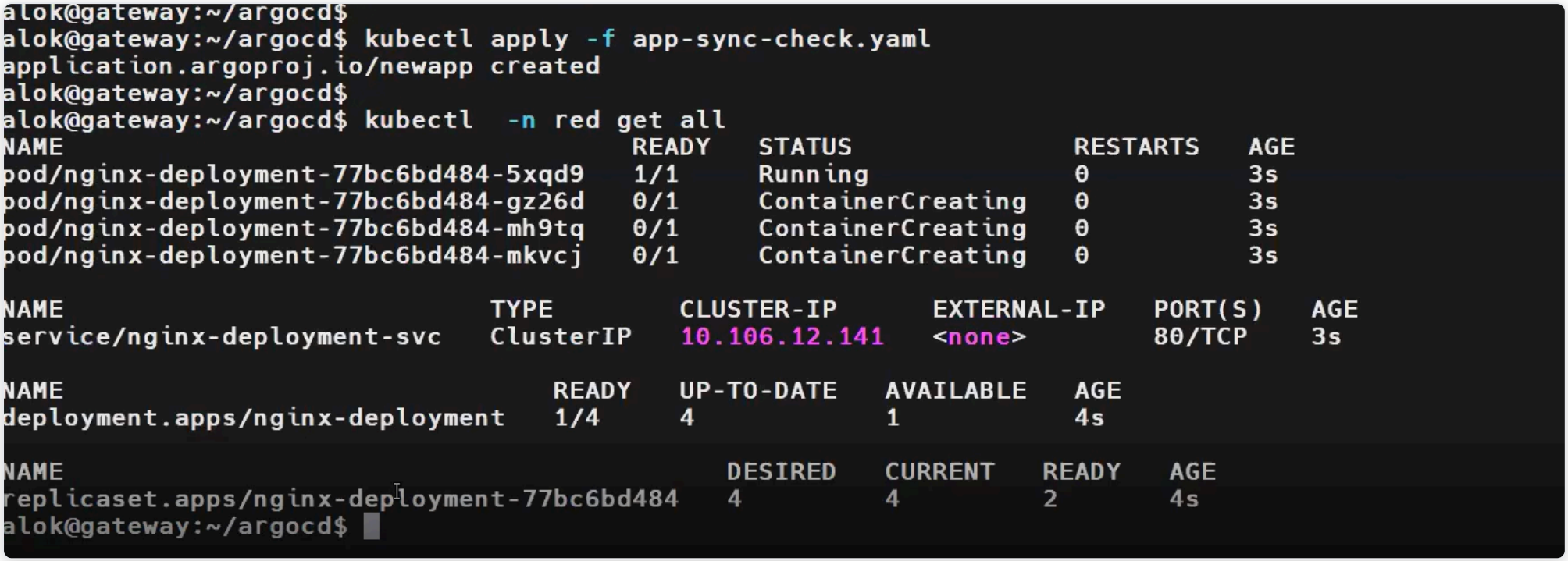
- Git repository changes occur.
- Argo CD refresh cycle detects drift.
- Manual refresh happens.


namespace: red # target namespace where resources will be d
syncPolicy:
automated: {} # Enables automated sync and self-healing


alok@gateway:~/argocd$
alok@gateway:~/argocd$
alok@gateway:~/argocd$ kubectl apply -f app-sync-check.yaml
application.argoproj.io/newapp created
alok@gateway:~/argocd$


d: {}


alok@gateway:~/argocd$ kubectl -n red get all
NAME READY STATUS RES
pod/nginx-deployment-77bc6bd484-5xqd9 1/1 Running 0
pod/nginx-deployment-77bc6bd484-gz26d 0/1 ContainerCreating 0
pod/nginx-deployment-77bc6bd484-mh9tq 0/1 ContainerCreating 0
pod/nginx-deployment-77bc6bd484-mkvcj 0/1 ContainerCreating 0


alok@gateway:~/argocd$
alok@gateway:~/argocd$ kubectl apply -f app-sync-check.yaml
application.argoproj.io/newapp created
alok@gateway:~/argocd$
alok@gateway:~/argocd$ kubectl -n red get all
NAME READY STATUS RESTARTS AGE
pod/nginx-deployment-77bc6bd484-5xqd9 1/1 Running 0 3s
pod/nginx-deployment-77bc6bd484-gz26d 0/1 ContainerCreating 0 3s
pod/nginx-deployment-77bc6bd484-mh9tq 0/1 ContainerCreating 0 3s
pod/nginx-deployment-77bc6bd484-mkvcj 0/1 ContainerCreating 0 3s
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
service/nginx-deployment-svc ClusterIP 10.106.12.141 <none> 80/TCP 3s
NAME READY UP-TO-DATE AVAILABLE AGE
deployment.apps/nginx-deployment 1/4 4 1 4s
NAME DESIRED CURRENT READY AGE
replicaset.apps/nginx-deployment-77bc6bd484 4 4 2 4s
alok@gateway:~/argocd$

Automated Pruning

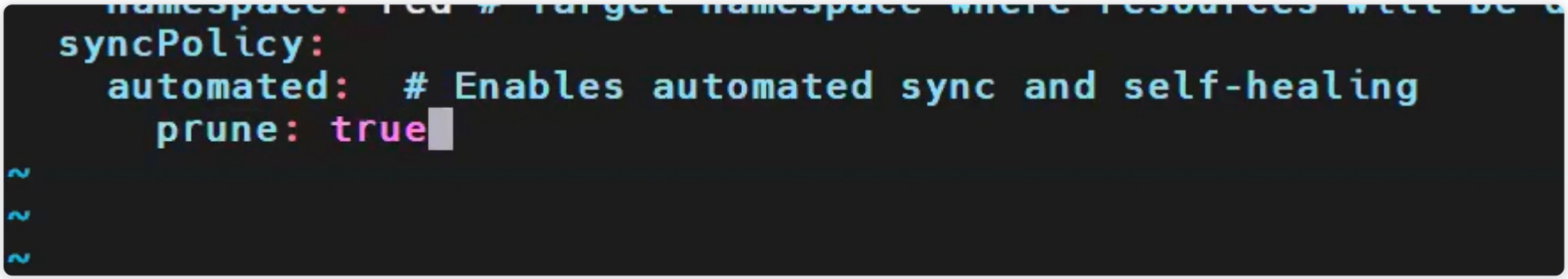
When automated sync enabled, by default it won't delete any resources when argocd detects the resources is no longer defined in Git.

Pruning can be enabled to delete resources automatically as part of the automated sync. Declarative:

```
syncPolicy:
   automated:
      prune: true
```

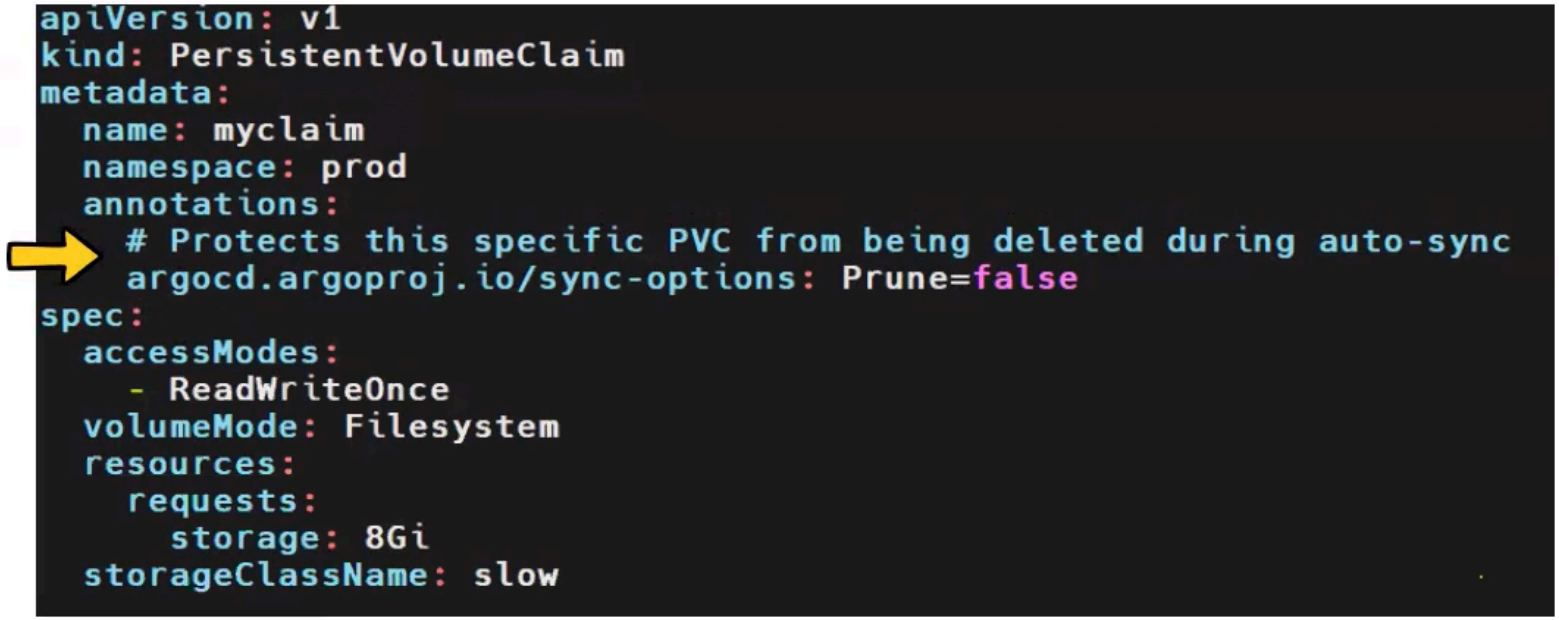
Pruning automatically removes:

- Resources deleted from Git.
- Orphaned Kubernetes objects.
- Resources no longer managed.


syncPolicy:
automated: # Enables automated sync and self-healing
prune: true

Argo CD Sync Policies - Options

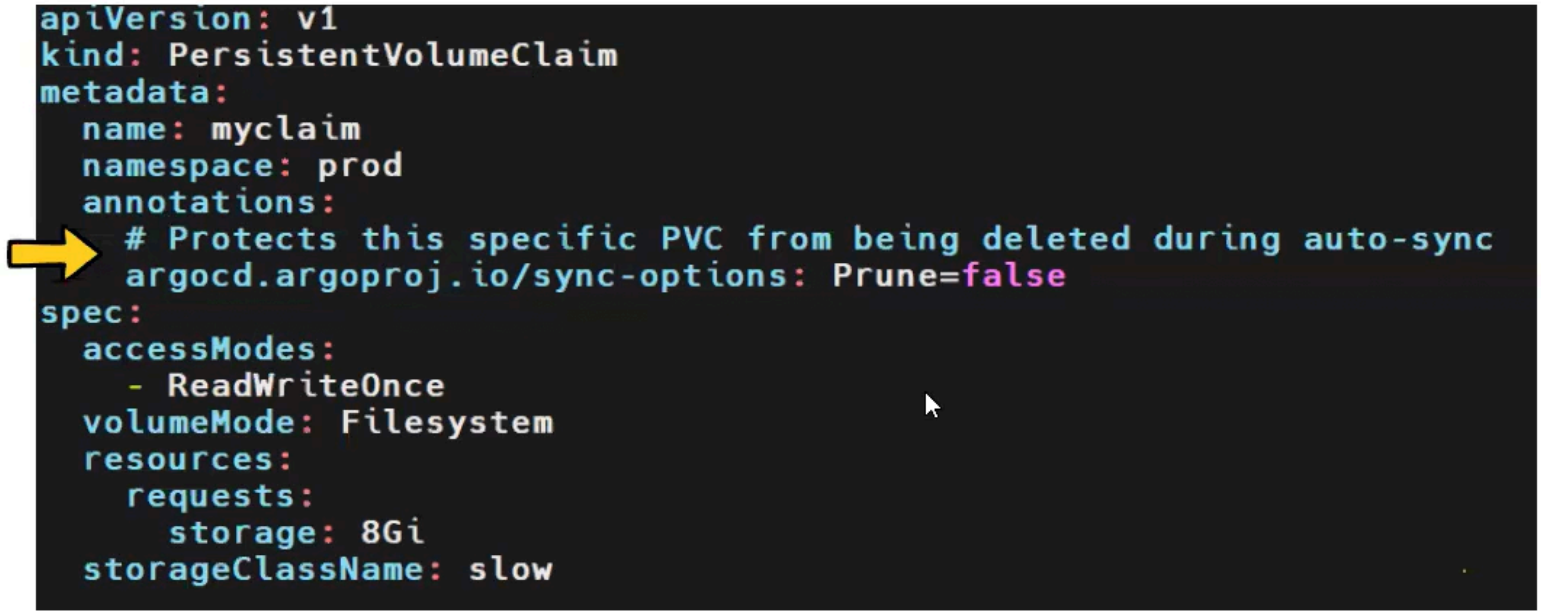
You may wish to prevent an object from being pruned:



```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: myclaim
  namespace: prod
  annotations:
    # Protects this specific PVC from being deleted during auto-sync
    argocd.argoproj.io/sync-options: Prune=false
spec:
  accessModes:
    - ReadWriteOnce
  volumeMode: Filesystem
  resources:
    requests:
      storage: 8Gi
  storageClassName: slow
```

Argo CD Sync Policies - Options

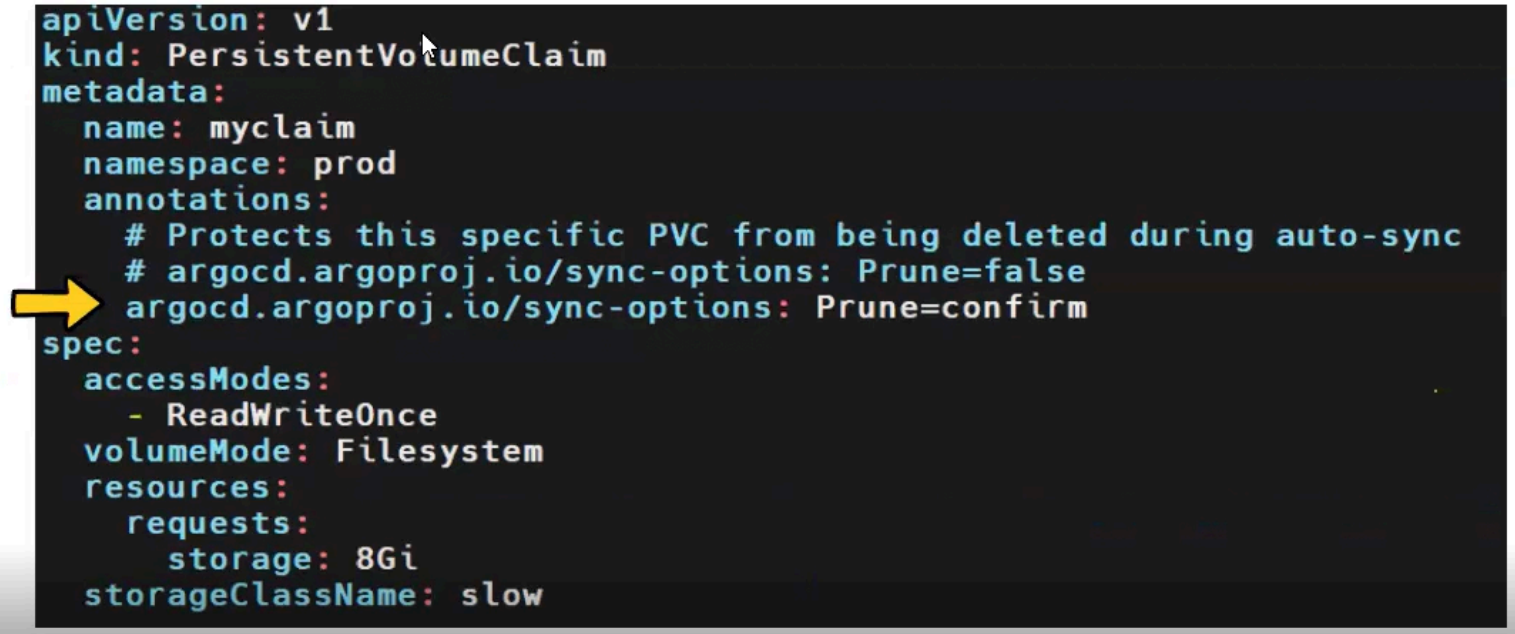
You may wish to prevent an object from being pruned:



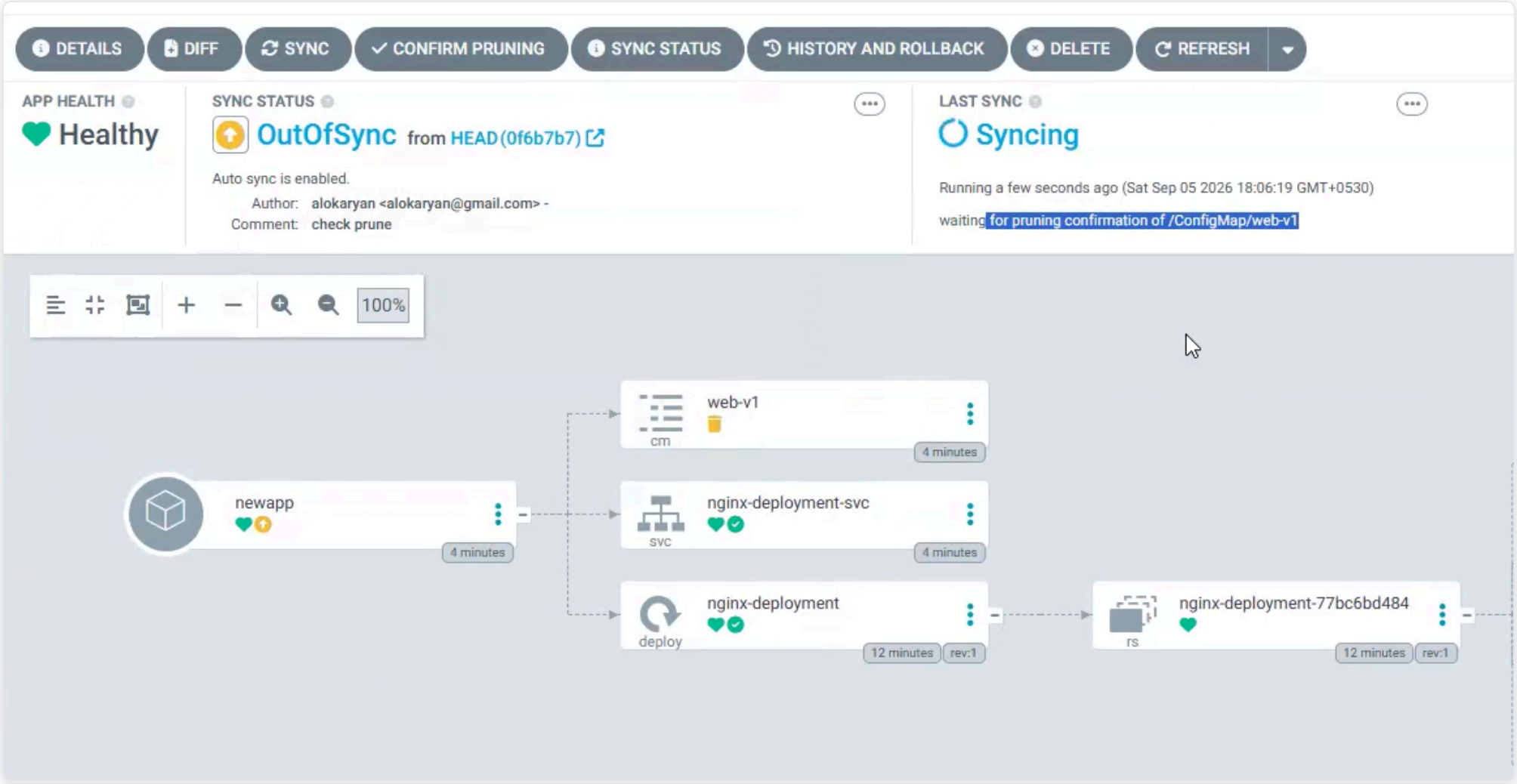
```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: myclaim
  namespace: prod
  annotations:
    # Protects this specific PVC from being deleted during auto-sync
    argocd.argoproj.io/sync-options: Prune=false
spec:
  accessModes:
    - ReadWriteOnce
  volumeMode: Filesystem
  resources:
    requests:
      storage: 8Gi
  storageClassName: slow
```

Pruning with Confirmation

Resources such as Namespaces / PVC/ PV etc are critical and should not be pruned without confirmation. You can set the Prune=confirm sync option to require manual confirmation before pruning.



```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: myclaim
  namespace: prod
  annotations:
    # Protects this specific PVC from being deleted during auto-sync
    # argocd.argoproj.io/sync-options: Prune=false
    argocd.argoproj.io/sync-options: Prune=confirm
spec:
  accessModes:
    - ReadWriteOnce
  volumeMode: Filesystem
  resources:
    requests:
      storage: 8Gi
  storageClassName: slow
```


DETAILS
DIFF
SYNC
CONFIRM PRUNING
SYNC STATUS
HISTORY AND ROLLBACK
DELETE
REFRESH
APP HEALTH
Healthy
SYNC STATUS
OutOfSync from HEAD (0f6b7b7)
Auto sync is enabled.
Author: alokaryan <alokaryan@gmail.com> -
Comment: check prune
LAST SYNC
Syncing
Running a few seconds ago (Sat Sep 05 2026 18:06:19 GMT+0530)
waiting for pruning confirmation of /ConfigMap/web-v1
100%
newapp
4 minutes
web-v1
cm
4 minutes
nginx-deployment-svc
svc
4 minutes
nginx-deployment
deploy
12 minutes
rev:1
nginx-deployment-77bc6bd484
rs
12 minutes
rev:1

Or you can set this option as a default option on the application level:

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: webapp
  namespace: argocd # The namespace where Argo CD is installed
  finalizers:       ## USE ONLY ONE OPTION
    - resources-finalizer.argocd.argoproj.io  # This enables foreground CASCADE delete
    # Alternatively, you can use background cascading deletion
    # - resources-finalizer.argocd.argoproj.io/background  # This enable background CASCADE delete
spec:
  project: default # Default project provided by Argo CD
  source:
    repoURL: 'https://github.com/networknuts/argocd.git' # Your Git repo URL
    targetRevision: HEAD # Git branch, tag, or commit hash
    path: ch-04 # Path within the repository containing the YAML files
  destination:
    server: 'https://kubernetes.default.svc' # Target cluster URL (this means the local cluster)
    namespace: dev # Target namespace where resources will be deployed
  syncPolicy:
    ### CORE SYNC POLICY
    automated: # Enables automated sync and self-healing
      prune: true # Deletes resources from K8s if removed from Git
      selfHeal: true # Overwrites manual changes made directly in the cluster
    ### SYNC OPTIONS
    syncOptions:
      - Prune=confirm
```

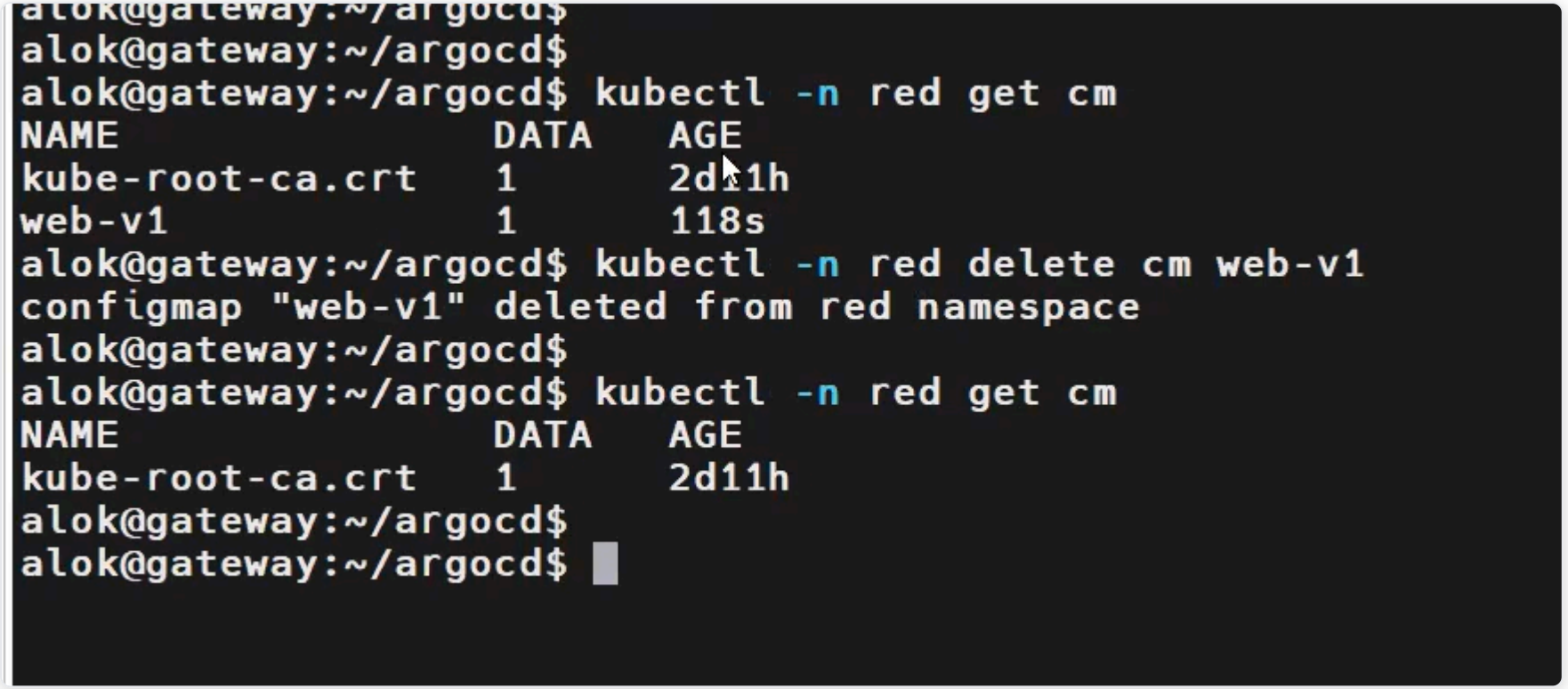
Self Healing

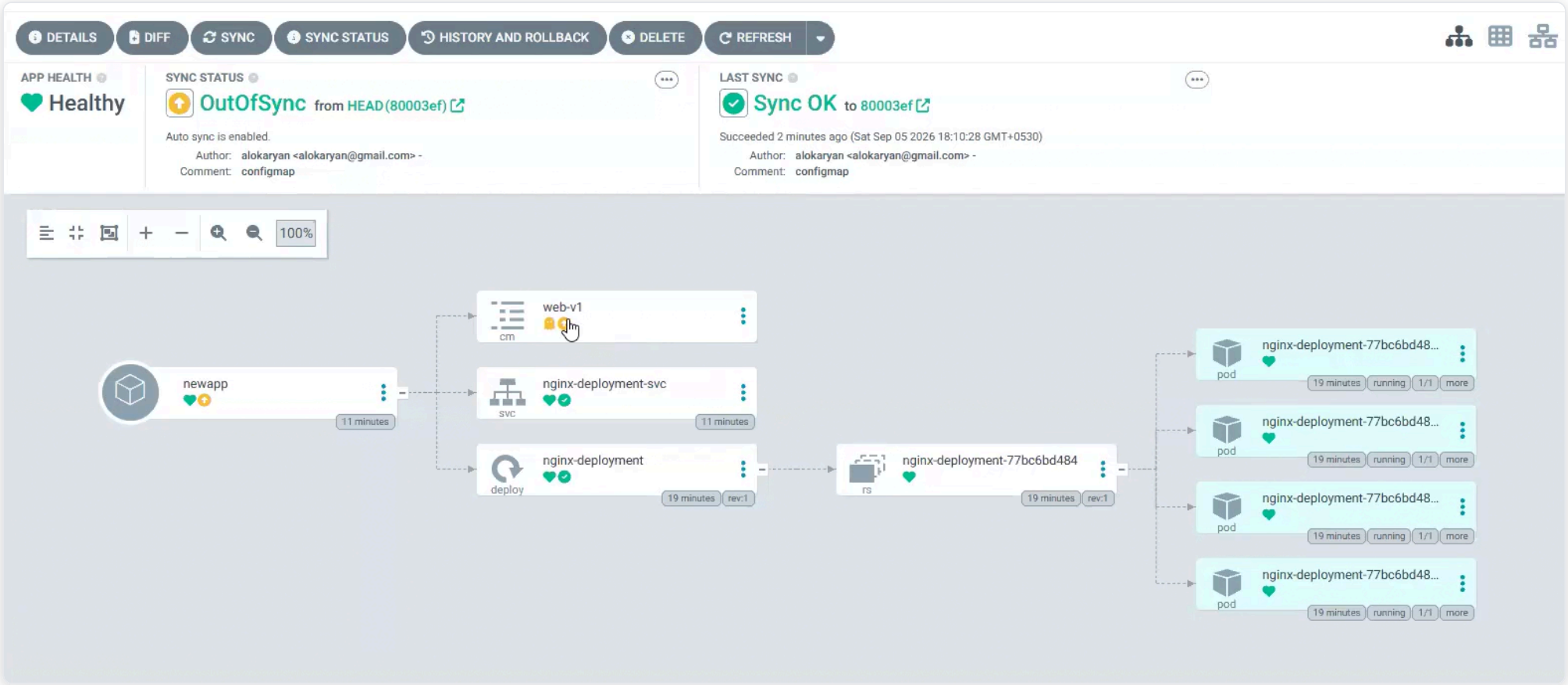
Self-healing is a feature that automatically reverts unauthorized or manual changes made directly to the live cluster.

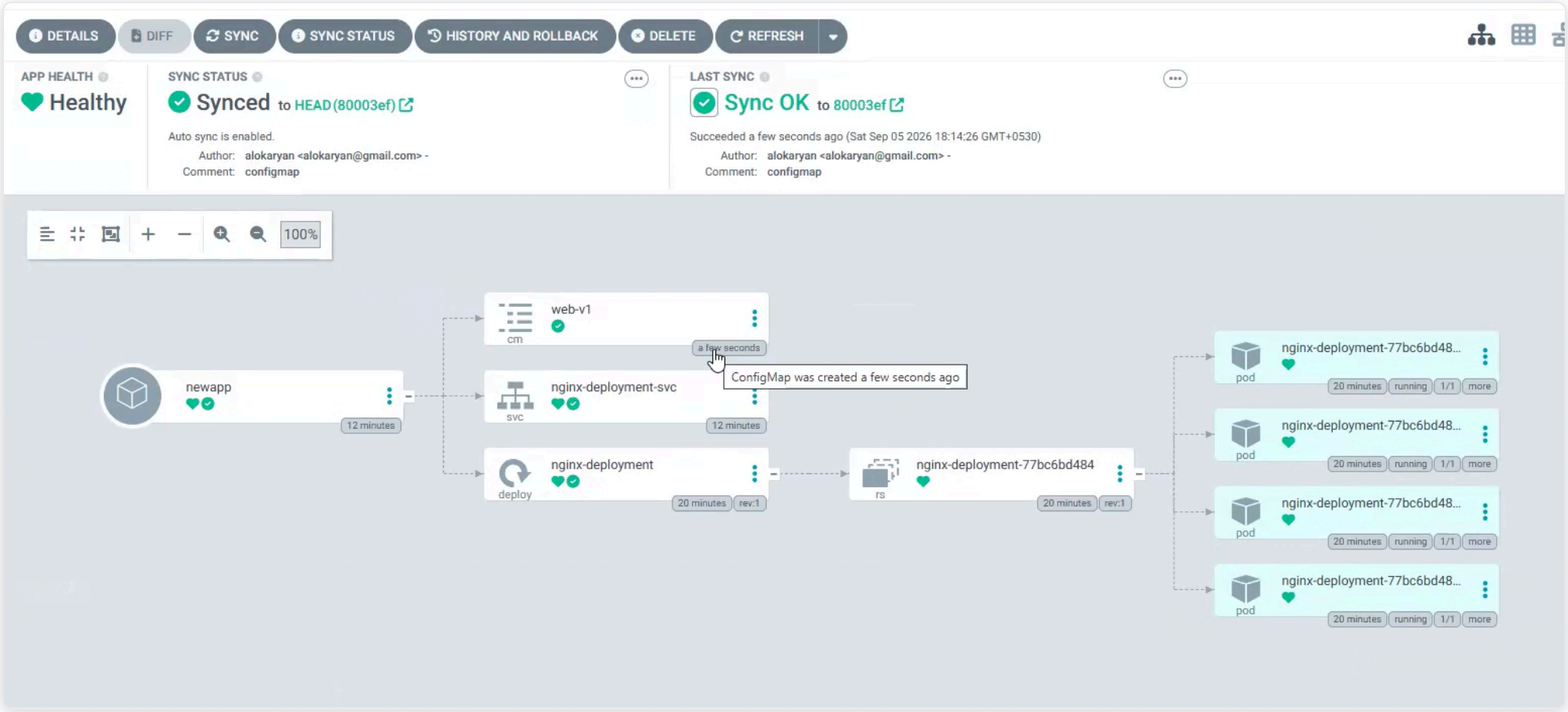
If a user modifies a live resource (e.g., using kubectl edit), ArgoCD detects the configuration drift and immediately overwrites it to match the desired state defined in your Git repository. Self-healing only functions when the automated sync policy (spec.syncPolicy.automated) is enabled.

Argo CD Sync Policies - Options

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: webapp
  namespace: argocd # The namespace where Argo CD is installed
  finalizers:       ## USE ONLY ONE OPTION
    - resources-finalizer.argocd.argoproj.io  # This enables foreground CASCADE delete
    # Alternatively, you can use background cascading deletion
    # - resources-finalizer.argocd.argoproj.io/background  # This enable background CASCADE delete
spec:
  project: default # Default project provided by Argo CD
  source:
    repoURL: 'https://github.com/networknuts/argocd.git' # Your Git repo URL
    targetRevision: HEAD # Git branch, tag, or commit hash
    path: ch-04 # Path within the repository containing the YAML files
  destination:
    server: 'https://kubernetes.default.svc' # Target cluster URL (this means the local cluster)
    namespace: dev # Target namespace where resources will be deployed
  syncPolicy:
    automated: # Enables automated sync and self-healing
      prune: true # Deletes resources from K8s if removed from Git
      selfHeal: true # Overwrites manual changes made directly in the cluster
```


alok@gateway:~/argocd$
alok@gateway:~/argocd$
alok@gateway:~/argocd$ kubectl -n red get cm
NAME DATA AGE
kube-root-ca.crt 1 2d11h
web-v1 1 118s
alok@gateway:~/argocd$ kubectl -n red delete cm web-v1
configmap "web-v1" deleted from red namespace
alok@gateway:~/argocd$
alok@gateway:~/argocd$ kubectl -n red get cm
NAME DATA AGE
kube-root-ca.crt 1 2d11h
alok@gateway:~/argocd$
alok@gateway:~/argocd$


DETAILS
DIFF
SYNC
SYNC STATUS
HISTORY AND ROLLBACK
DELETE
REFRESH
APP HEALTH
Healthy
SYNC STATUS
OutOfSync from HEAD (80003ef)
Auto sync is enabled.
Author: alokaryan <alokaryan@gmail.com> -
Comment: configmap
LAST SYNC
Sync OK to 80003ef
Succeeded 2 minutes ago (Sat Sep 05 2026 18:10:28 GMT+0530)
Author: alokaryan <alokaryan@gmail.com> -
Comment: configmap
100%
newapp
11 minutes
web-v1
cm
nginx-deployment-svc
svc
11 minutes
nginx-deployment
deploy
19 minutes
rev:1
nginx-deployment-77bc6bd484
rs
19 minutes
rev:1
nginx-deployment-77bc6bd48...
pod
19 minutes
running
1/1
more


DETAILS
DIFF
SYNC
SYNC STATUS
HISTORY AND ROLLBACK
DELETE
REFRESH
APP HEALTH
Healthy
SYNC STATUS
Synced to HEAD (80003ef)
Auto sync is enabled.
Author: alokaryan <alokaryan@gmail.com> -
Comment: configmap
LAST SYNC
Sync OK to 80003ef
Succeeded a few seconds ago (Sat Sep 05 2026 18:14:26 GMT+0530)
Author: alokaryan <alokaryan@gmail.com> -
Comment: configmap
100%
newapp
12 minutes
web-v1
cm
a few seconds
ConfigMap was created a few seconds ago
nginx-deployment-svc
svc
12 minutes
nginx-deployment
deploy
20 minutes
rev:1
nginx-deployment-77bc6bd484
rs
20 minutes
rev:1
nginx-deployment-77bc6bd48...
pod
20 minutes
running
1/1
more


01
GIT = CLUSTER STATE
Soure of Truth (Git) and Clsuter state are in Sync.
#networknuts
02
Out of Band Change
Manually modified resource using kubectl. Drift. Cluster≠Git. Sync status=OutOfSync.
ArgoCD
SELF HEALING
#networknuts
03
ArgoCD Control Plane
ArgoCD constantly monitor and reconcile. Self-healing mode enabled.
#networknuts
04
Cluster Corrected
Automatic sync & correction. Resources re-applied to match Git. Sync Status: Sync/Healthy

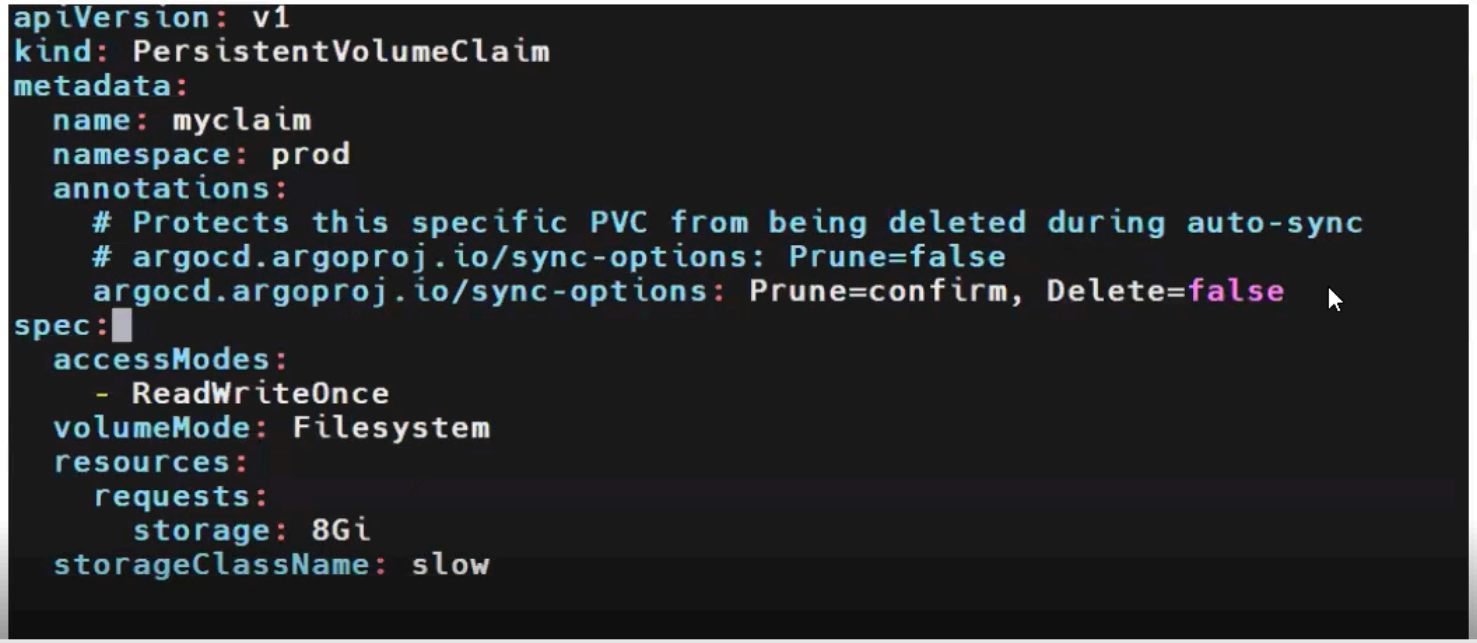
Retry

You can configure automatic retries for failed synchronization operations by defining the retry block inside the syncPolicy section of your Application manifest. This prevents transient cluster errors, network hiccups, or temporary API timeouts from breaking your GitOps pipeline.

```yaml
syncPolicy:
  automated: # Enables automated sync and self-healing
    prune: true # Deletes resources from K8s if removed from Git
    selfHeal: true # Overwrites manual changes made directly in the cluster
  retry:
    limit: 5 # Number of retries before giving up (-1 for unlimited)
    backoff:
      duration: 5s # Initial wait time before the first retry
      factor: 2 # Multiplier applied to duration after each failure
      maxDuration: 3m # The maximum allowable wait time between retries
    refresh: true # Optional: forces a Git refresh on new commits while retrying
```

No Resource Deletion

You might want to retain some resources even after your application is deleted, e.g. for Persistent Volume Claims. In such situations you can stop those resources from being cleaned up during app deletion by using the following annotation:



```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: myclaim
  namespace: prod
  annotations:
    # Protects this specific PVC from being deleted during auto-sync
    # argocd.argoproj.io/sync-options: Prune=false
    argocd.argoproj.io/sync-options: Prune=confirm, Delete=false
spec:
  accessModes:
    - ReadWriteOnce
  volumeMode: Filesystem
  resources:
    requests:
      storage: 8Gi
  storageClassName: slow
```

Argo CD Sync Policies - Advance Options

No Resource Deletion

Set this option as a default option on the application level:

```yaml
syncPolicy:
  ### CORE SYNC POLICY
  automated: # Enables automated sync and self-healing
    prune: true # Deletes resources from K8s if removed from Git
    selfHeal: true # Overwrites manual changes made directly in the cluster
  retry:
    limit: 5
    backoff:
      duration: 5s
      factor: 2
      maxDuration: 3m
    refresh: true
  ### SYNC OPTIONS
  syncOptions:
    - Prune=confirm
    - Delete=confirm
    - CreateNamespace=true
```

Argo CD Sync Policies - Which is Best

Choosing the Right Policy

Development environments, use full automation for rapid iteration:

```
syncPolicy:
  automated:
    prune: true
    selfHeal: true
```

Staging environments, use automation with manual pruning:

```
syncPolicy:
  automated:
    prune: false
    selfHeal: true
```